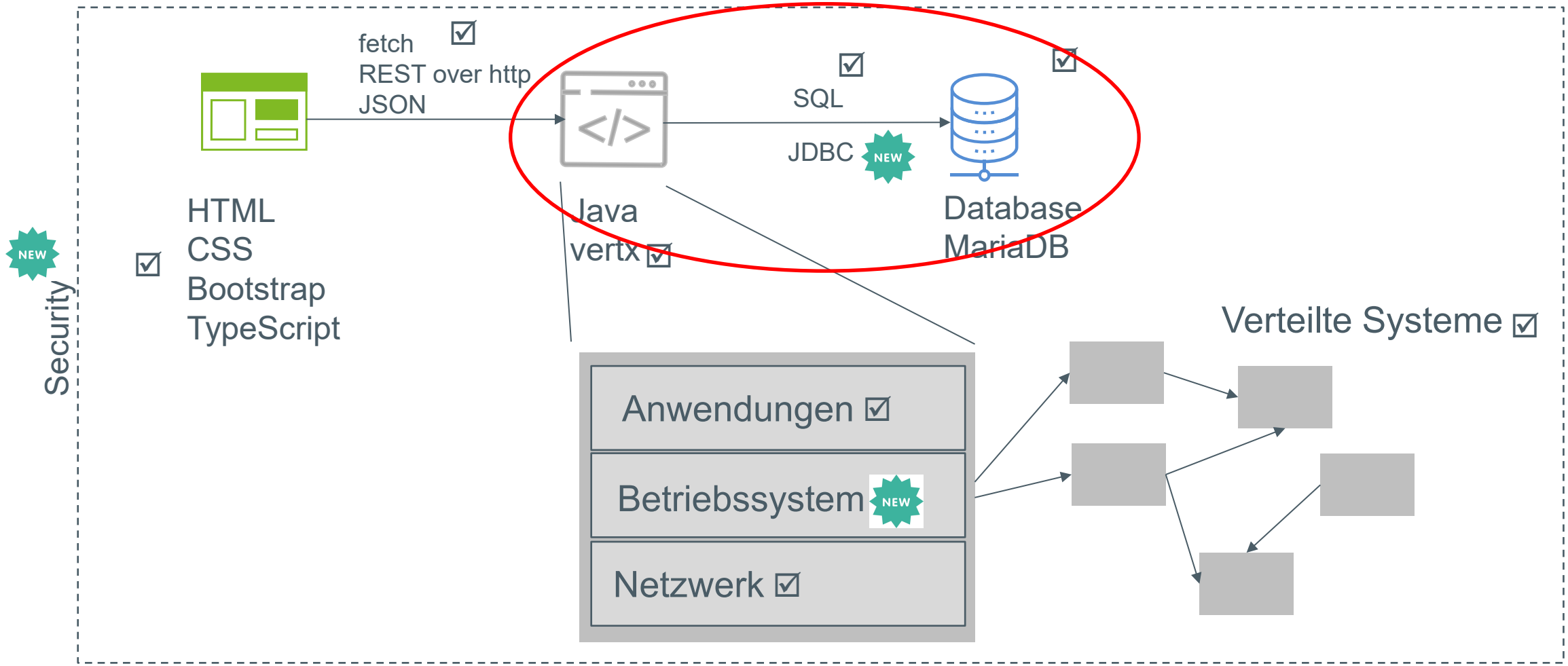
Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

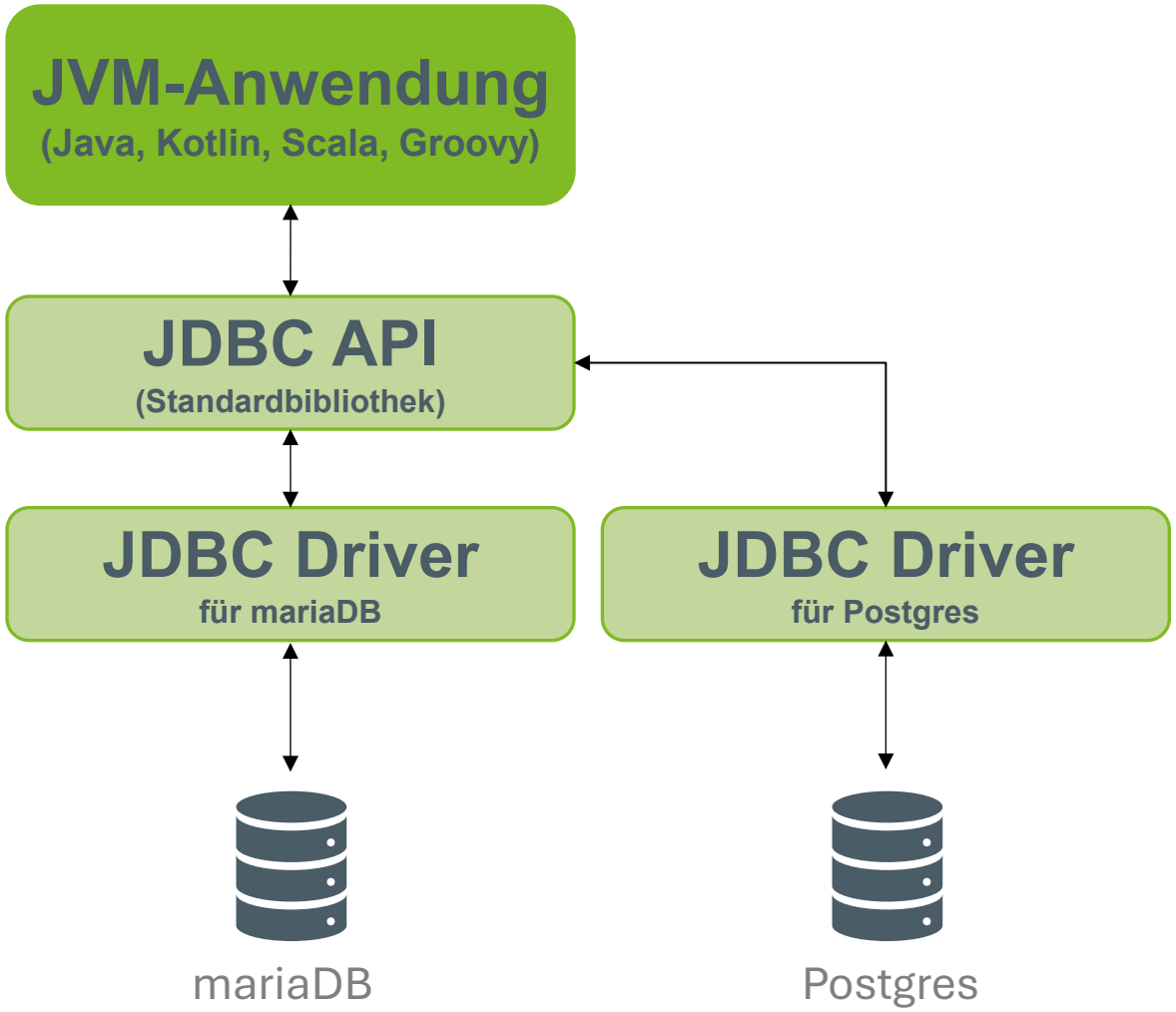
Zwischenstand



Web-Anwendungen mit Datenbank

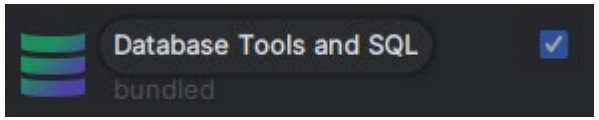
- Verwendung von SQL als String
- Gleichzeitiger Zugriff auf mehrere Datenbanken
- Grundlage für andere Techniken, wie:

  SQLJ, JPA, Exposed
- Ziele:
  - Einfachheit
  - Robustheit
  - Verfügbarkeit
  - Skalierbarkeit
- Das JDBC API ist generisch; der Treiber spezifisch



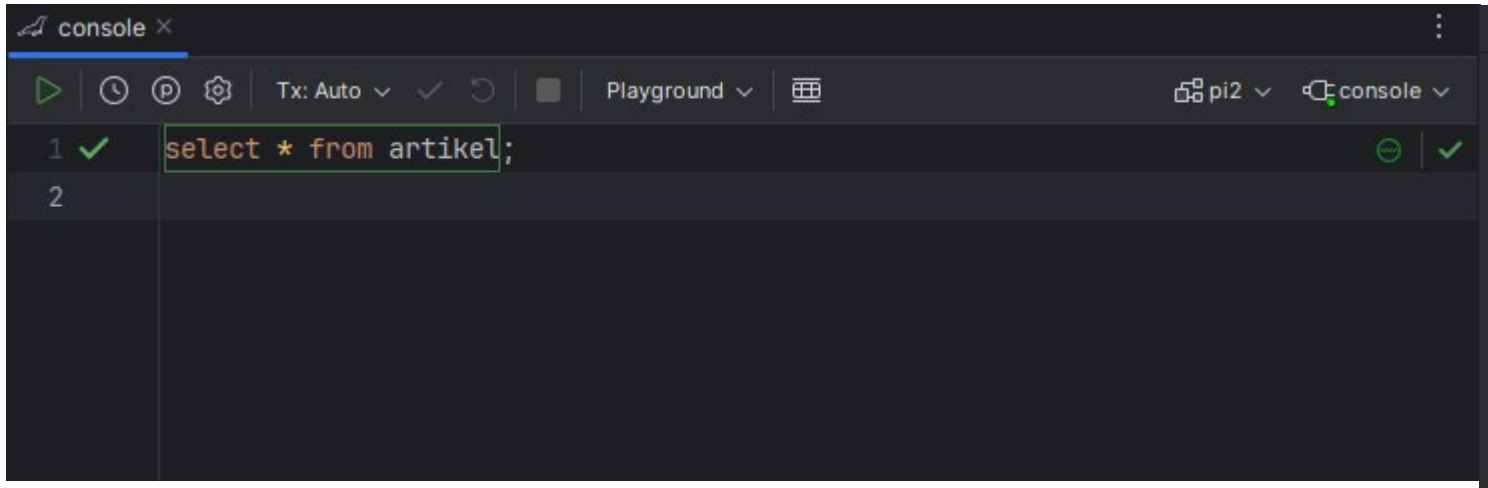
Nutzen mariaDB mit IntelliJ

- IntelliJ ist eine IDE für Java und stammt ebenfalls von JetBrains (wie WebStorm)
- Über die JetBrains Toolbox können Sie sich IntelliJ installieren (Lizenz über Ihre THM Email-Adresse)
- Legen Sie ein Java-Projekt an
- Verifizieren Sie, dass das plugin „Database Tools and SQL“ bei Ihnen installiert ist.



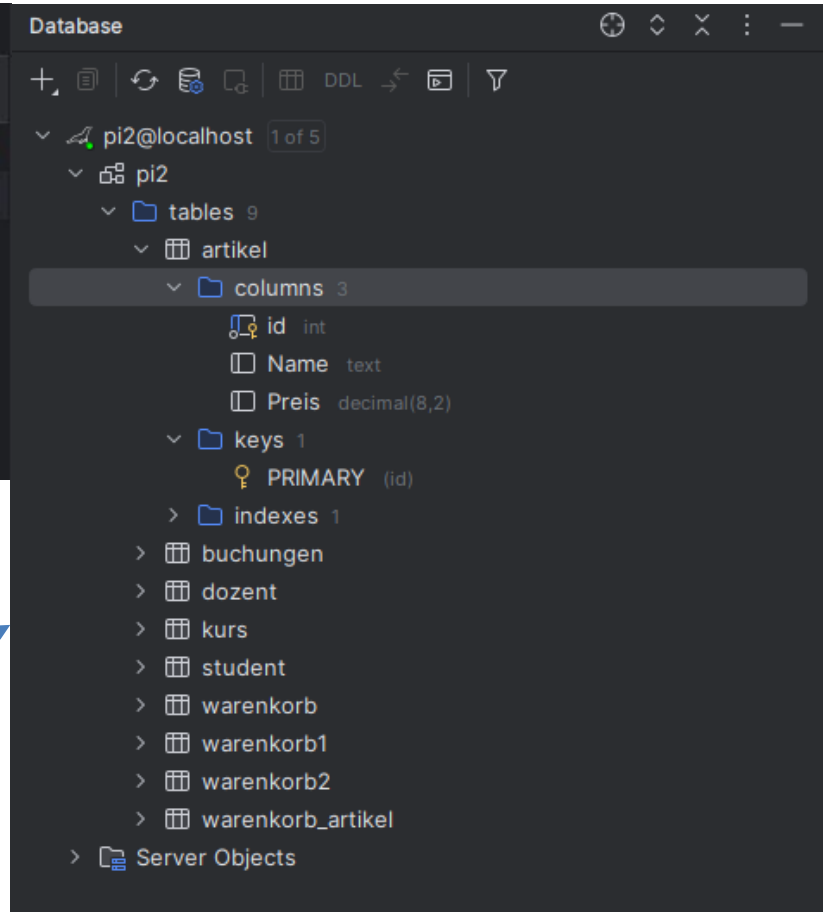
- Verbinden Sie sich mit Ihrer mariaDB-Instanz:
  - File -> New -> Data Source -> MariaDB
  - Auf der folgenden Maske geben Sie den Namen Ihres Datenbankbenutzers und dessen Passwort ein
  - Selektieren Sie die Datenbank (z.B. pi2).
  - Ggf. „Download missing drivers“ unten anklicken
  - „Test Connection“
    Hier sollte nach einigen Sekunden „Succeeded“ stehen.

Database Plugin

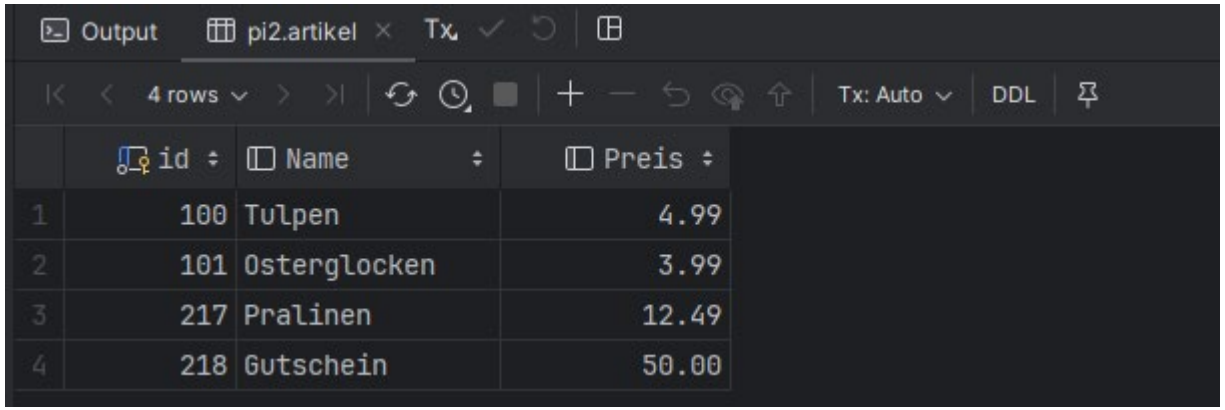


Eingabefenster für SQL-Befehle

Struktur der Datenbank



Database Plugin

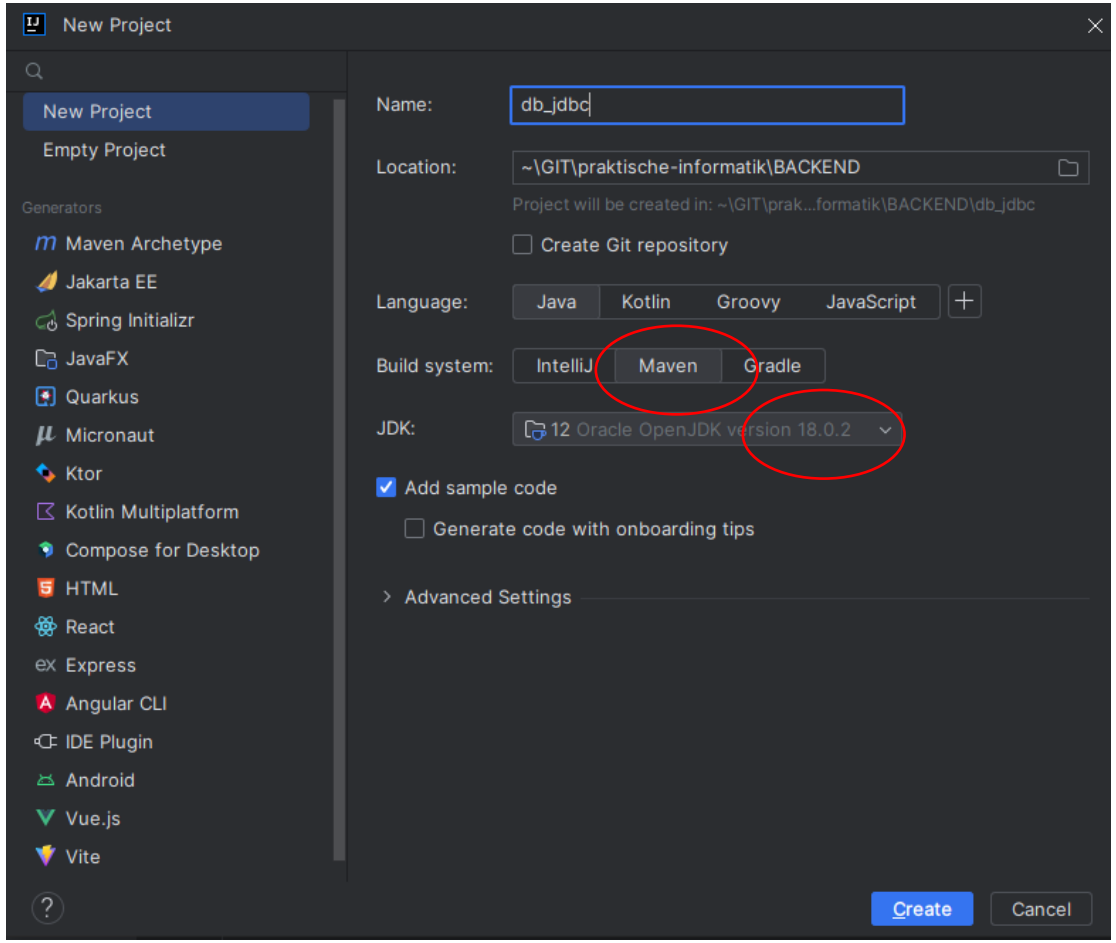


	id	Name	Preis
1	100	Tulpen	4.99
2	101	Osterglocken	3.99
3	217	Pralinen	12.49
4	218	Gutschein	50.00

Ausgabe der Ergebnisse

Wir schreiben eine jdbc-Anwendung … (1)

- Starten Sie IntelliJ und erstellen Sie ein neues Java-Projekt mit
  - Build System: Maven
  - JDK mindestens 18.0.2
- Verbinden Sie sich mit Ihrer mariaDB-Instanz

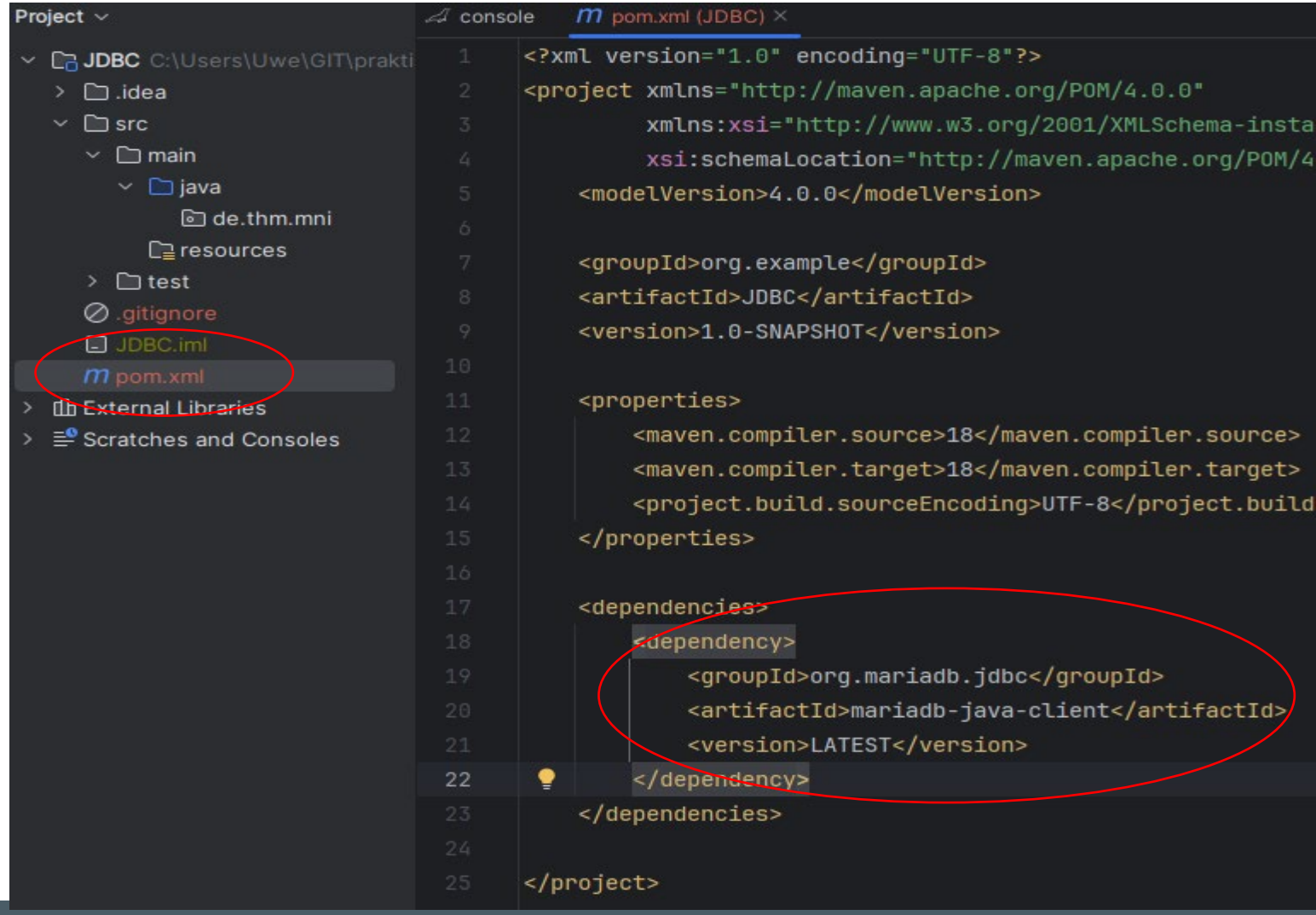


Wir schreiben eine jdbc-Anwendung … (2)

- Öffnen Sie die Datei `pom.xml` und fügen Sie den mariaDB-jdbc-Treiber hinzu.

Erklärung:

- Diese Datei enthält die Konfiguration für das Build-System Maven.
- Maven „weiß“ jetzt, dass es die jar-Datei mit dem Treiber von dem Server „Maven Central“ herunterladen und diese beim Bauen und Ausführen in den CLASSPATH nehmen muss.



Wir schreiben eine jdbc-Anwendung … (3)

- Importiere `java.sql.*`

- Öffne eine Verbindung zur Datenbank

```
try (Connection connection = DriverManager.getConnection(
        "jdbc:mariadb://localhost:3306/pi2",
        "user1", "passwort")) {

…
} catch (SQLException sqlex) { sqlex.printStackTrace(); }
```

- Statement erzeugen:

```
String query = "SELECT * from artikel";
Statement stmt = connection.createStatement();
```

Wir schreiben eine jdbc-Anwendung … (4)

- Statement ausführen

```
ResultSet rs = stmt.executeQuery(query);
```

- Über das ResultSet iterieren und Ergebnisse abfragen

```
while(rs.next()) {
    int id                  = rs.getInt("id");
    String text             = rs.getString("Name");
    BigDecimal preis = rs.getBigDecimal("Preis");

    System.out.println(id + ", " + text+", " + preis);
}
```

```
package de.thm.mni;
import java.sql.*;
import java.math.BigDecimal;
public class TestConnection {
    public static void main(String[] args) {

        // Öffne die Connection der Datenbank mit einem try-with-resources
        try (Connection connection = DriverManager.getConnection(
                "jdbc:mariadb://localhost:3306/pi2",
                "user1", "passwort")) {

            // query the artikel table
            String query = "SELECT * from artikel";
            Statement stmt = connection.createStatement();
            ResultSet rs = stmt.executeQuery(query);
            while(rs.next()) {
                int id = rs.getInt("id");
                String text = rs.getString("Name");
                BigDecimal preis = rs.getBigDecimal("Preis");
                System.out.println(id + ", " + text+", " + preis);
            }
        } catch (SQLException sqlex) {
            sqlex.printStackTrace();
        }
    }
}
```

Prepared-Statement

- Bei der Methode eben wird das Statement erst (in Java) zusammengesetzt und dann im Ganzen an die Datenbank geschickt.
- Besser und sicherer (!) sind sogenannte **Prepared-Statements**:

```
PreparedStatement pstmt = connection.prepareStatement("select * from artikel r where r.name=?;");

// search for Tulpen
pstmt.setString(1, "Tulpen");
ResultSet rs = pstmt.executeQuery();
```

- Vorteile:
  - Schnellere Ausführung gleichartiger Abfragen
  - Sauberer Code
    - Statt String-Verkettung werden Variablen eingesetzt
    - Das Statement wird nur an einer Stelle „gebaut“
  - Keine SQL-Injection möglich (siehe Kapitel 8)

Adaptives Programmieren mit JDBC / Metadaten

Über Interfaces, kann die Struktur der DB (Schema) abgefragt werden

ResultSetMetaData → Informationen über die Ergebnismenge

- int getColumnCount()
- String getColumnName(int column)
- int getColumnType(int column)

DatabaseMetaData → Informationen über das DBMS und das Schema

- String getDatabaseProductName()
- ResultSet getTables(…)
- ResultSet getColumns(…, String tableName)
- String getTimeDateFunctions()
- … und sehr viele mehr.

Beispiel: DESCRIBE mit JDBC

```
private static void describe(Connection conn, String tableName) {

    try {
        DatabaseMetaData metaData = conn.getMetaData();
        ResultSet columns = metaData.getColumns(null, null, tableName, null);

        System.out.printf("%-15s %-15s %-15s\n", "Column Name", "Data Type",
"Constraints");
        System.out.println("-------------------------------------------------");

        while (columns.next()) {
            String columnName = columns.getString("COLUMN_NAME");
            String dataType = columns.getString("TYPE_NAME");
            String constraints = columns.getString("COLUMN_DEF");

            System.out.printf("%-15s %-15s %-15s\n", columnName, dataType, constraints);
        }
    } catch (SQLException e) {
        e.printStackTrace();
        System.exit(-1);
    }
}
```

Datentypen von SQL → JDBC

SQL Typ	Java Typ
INT, INTEGER, SERIAL	int
VARCHAR	String
CHAR	String
TEXT	String
DECIMAL, NUMERIC	java.math.BigDecimal
FLOAT, DOUBLE PRECISION	double
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp
BLOB, CLOB	byte[]

Datenmanipulation mit JDBC

Einfachstes Vorgehen: executeUpdate()

- UPDATE / INSERT wird als (Prepared)Statement mit `executeUpdate()` abgesetzt
- `executeUpdate()` liefert dann kein ResultSet, sondern die Anzahl der veränderten Zeilen

```
PreparedStatement pstmt =
conn.prepareStatement("insert into values (?,?,?);");

pstmt.setInt(1, 123);                    // set id
pstmt.setString(2, "Cheeseburger");      // set name
pstmt.setDouble(3, 4.99);                // set preis

int rowsInserted = pstmt.executeUpdate();
if (rowsInserted > 0) {
    System.out.println("Insert erfolgreich");
}
```

JDBC - ResultSet

- Navigation:
  - `next()`: Liefert `true` / `false` und springt zur nächsten Zeile
  - previous(): Wie `next()`, springt aber zurück
  - `first()` / `last()`
  - `beforeFirst()` / `afterLast()`
  - `relative(int rows)` / `absolute(int r)`

Im Standardfall:

```
ResultSet rs = pstmt.executeQuery();
while(rs.next()) {
    int id = rs.getInt("id");
    String text = rs.getString("Name");
    …
}
```

JDBC - ResultSet

Datenzugriff auf `ResultSet` ist immer auf eine Zeile der Ergebnismenge.

Werte lesen

- `rs.getString(1)` `// Nummer der Spalte (ab 1 !)`
- `rs.getString(„name“)` `// Name der Spalte`

Werte ändern

- `rs.updateBoolean(„privat“, true)`
- `rs.updateRow()`

Datensätze einfügen

- `rs.moveToInsertRow()`
- `rs.updateString(„name“, „Hoppenstedt“)`
- …
- `rs.insertRow()`